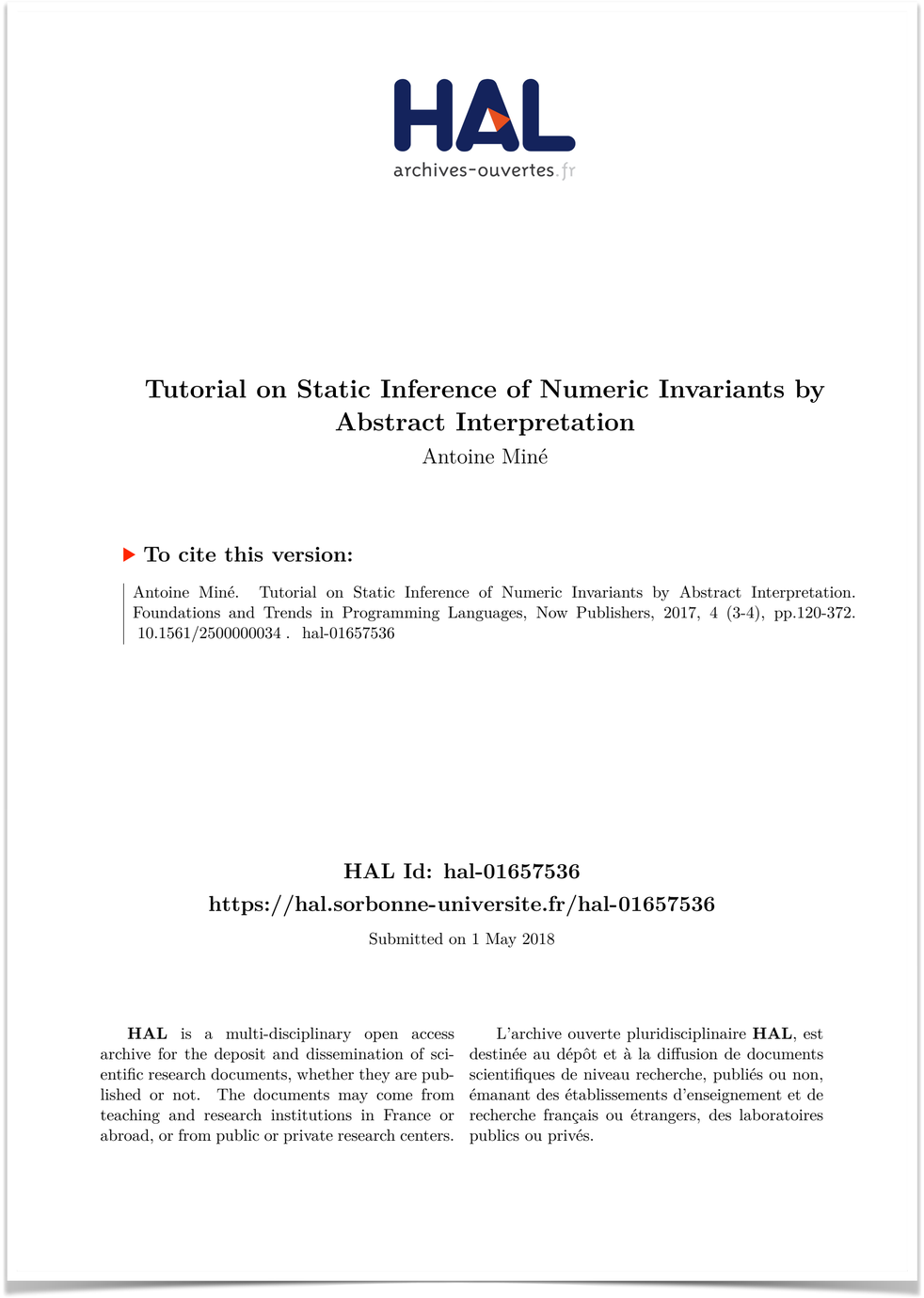
Program Analysis 2025-26

Lecture #15

Abstract analysis

Università di Pisa



HAL
archives-ouvertes.fr

Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation

Antoine Miné

Chapter 4:
Non-relational abstract domains

constant

constant set

intervals

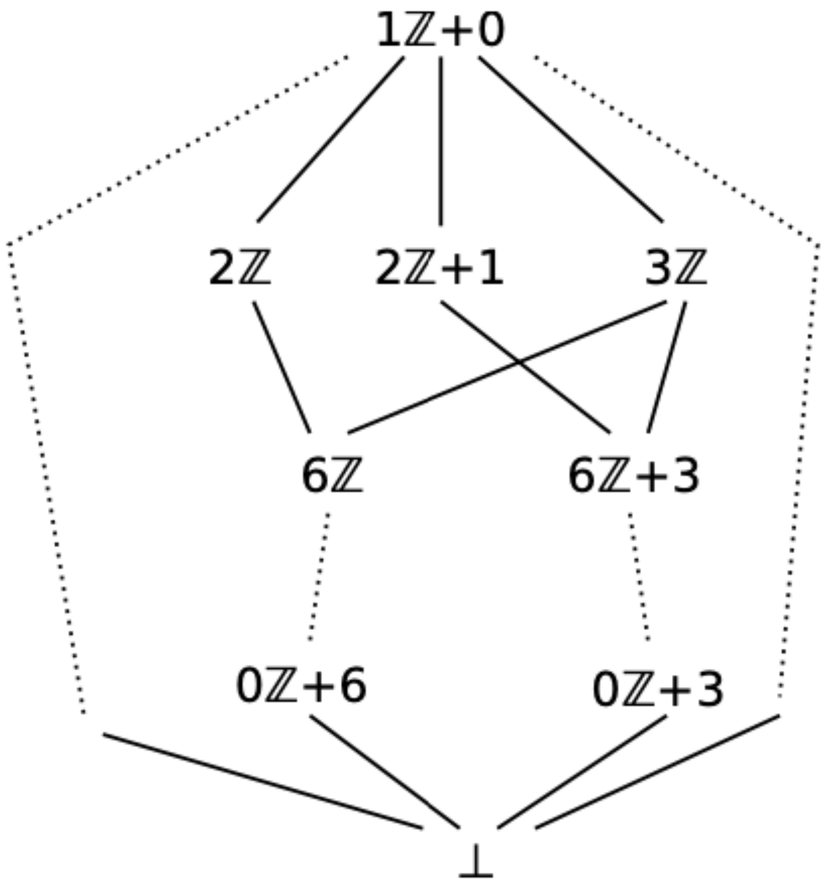
one more: congruence

Congruence domain

4.8 The Congruence Domain

The last non-relational abstract domain we present is a congruence domain, specific to integers, introduced by Granger [1989]. We thus assume $\mathbb{I} = \mathbb{Z}$, and infer value abstractions of the form $X \in a\mathbb{Z} + b$, meaning that X equals some multiple of a plus b. This domain is particularly useful to track pointer offsets in arrays and data-structures, prove alignment properties, or infer array access patterns.

Congruence domain



abstract elements of the form $a\mathbb{Z} + b$

(with $a \in \mathbb{N}$, $b \in \mathbb{Z}$)

$\gamma(\ \bot\ ) = \varnothing$

$\gamma(a\mathbb{Z}+b) = \{ak+b \mid k \in \mathbb{Z}\}$ multiples of a with offset b

e.g.

$\gamma(2\mathbb{Z}+0) =$ set of even numbers

$\gamma(2\mathbb{Z}+1) =$ set of odd numbers

$\gamma(0\mathbb{Z}+42) = \{42\}$

Congruence: concretization

γ is not injective

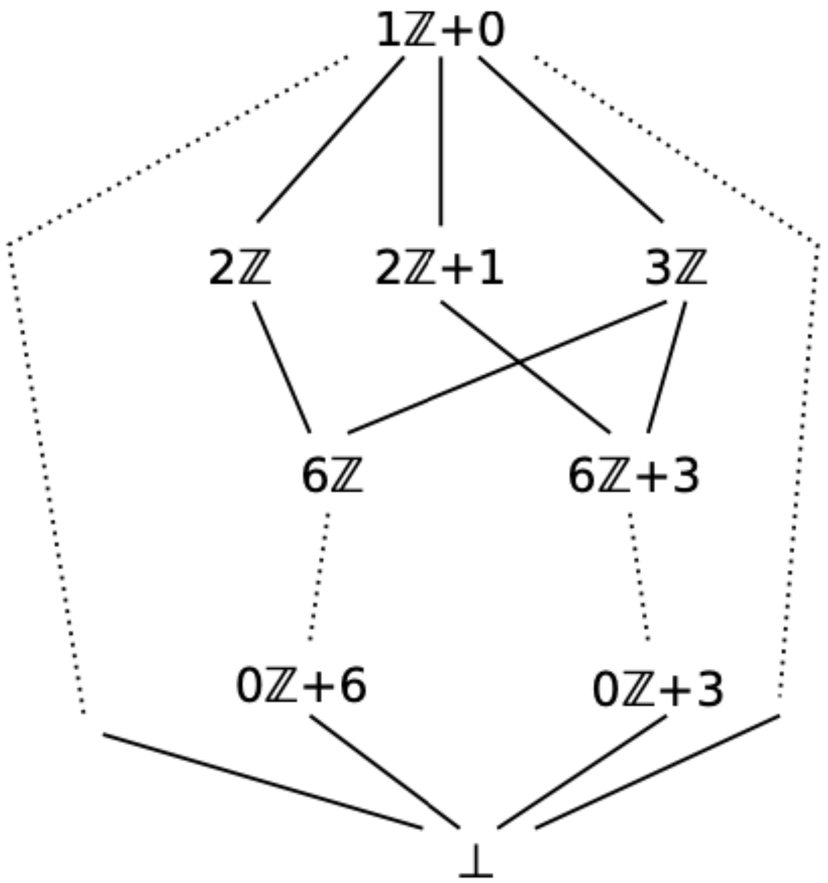
$$\gamma(2\mathbb{Z}+1) = \gamma(2\mathbb{Z}+3) = \gamma(2\mathbb{Z}-1)$$

$\gamma(a\mathbb{Z}+b) = \gamma(a\mathbb{Z}+(b+ak))$ for any $k \in \mathbb{Z}$

sometimes is assumed $0 \leq b < a$ (when $a \neq 0$)

then γ becomes injective

Congruence domain



- the greatest element is $1\mathbb{Z} + 0$
- precise on singletons:
  singleton $\{c\}$ is represented as $0\mathbb{Z} + c$
- $a\mathbb{Z} + b \sqsubseteq a'\mathbb{Z} + b'$ iff $\gamma(a\mathbb{Z} + b) \subseteq \gamma(a'\mathbb{Z} + b')$
- the bottom element is explictly added

Meet

Precise!

$$(a\mathbb{Z}+b) \sqcap^{\#} (a'\mathbb{Z}+b') = \begin{cases} \bot & \text{if } b \not\equiv b' \pmod{\gcd(a,a')} \\ a''\mathbb{Z}+b'' & \text{otherwise} \end{cases}$$

greatest common divisor

least common multiple

where $a'' = \mathrm{lcm}(a,a')$ and b'' satisfies $\begin{cases} b'' \equiv b \pmod{a} \\ b'' \equiv b' \pmod{a'} \end{cases}$

e.g. $4\mathbb{Z}+1 \sqcap^{\#} 6\mathbb{Z}+3$

$\gcd(4,6) = 2$ $\quad 1 \equiv 3 \bmod 2$ Compatible!

$a'' = \mathrm{lcm}(4,6) = 12$

$b'' \equiv 1 \bmod 4$
$b'' \equiv 3 \bmod 6$ $\longrightarrow$ $b'' = 9$ $\quad 12\mathbb{Z}+9$

Join

$$(a\mathbb{Z} + b) \sqcup^{\#} (a'\mathbb{Z} + b') = d\mathbb{Z} + r$$

$$\text{where } d = \gcd(a, a', b - b') \text{ and } r \equiv b \pmod{d}$$

e.g. $4\mathbb{Z} + 1 \sqcup^{\#} 4\mathbb{Z} + 3$

$d = gcd(4,4,2)$, $r = 1 \ mod \ 2$ $\qquad$ $2\mathbb{Z} + 1$

Congruence domain structure

$$\gamma(\ \bot\ ) = \varnothing$$

$$\gamma(a\mathbb{Z} + b) = \{ak + b \mid k \in \mathbb{Z}\}$$

$$\alpha(X) = \sqcup^{\#}_{x \in X}\, 0\mathbb{Z} + x$$

Although the domain has an infinite height, there is no infinite strictly increasing chain!

Why?

- moving up in the lattice = losing modular constraints
- losing constraints = the modulus a decreases
- but it cannot decrease indefinitely

$$6\mathbb{Z} \sqsubset 3\mathbb{Z} \sqsubset 1\mathbb{Z}$$

BCAs

$$-^{\#}(a\mathbb{Z}+b) = (a\mathbb{Z}+(-b))$$

When we sum $(a\mathbb{Z}+b)$ and $(c\mathbb{Z}+d)$ we have

$b+d+ka+lc$

$= b+d+k(k_1\, gcd(a,c))+l(l_1\, gcd(a,c))$

$$(a\mathbb{Z}+b) +^{\#} (c\mathbb{Z}+d) = (\gcd(a,c)\mathbb{Z}+(b+d))$$

$$(a\mathbb{Z}+b) -^{\#} (c\mathbb{Z}+d) = (\gcd(a,c)\mathbb{Z}+(b-d))$$

$$(a\mathbb{Z}+b) \times^{\#} (c\mathbb{Z}+d) = (\gcd(ac,ad,bc)\mathbb{Z}+bd)$$

Example

Example 4.11 (Congruence analysis)**.** *Consider the program:*

$X \leftarrow 0; Y \leftarrow 2;$
while $X < 40$ **do**
  $X \leftarrow X + 2;$
  if $X < 5$ **then** $Y \leftarrow Y + 18$ **endif**;
  if $X > 8$ **then** $Y \leftarrow Y - 30$ **endif**
done

inferred loop invariant:

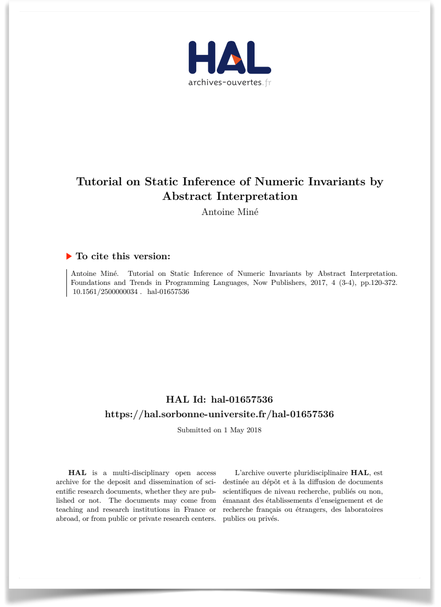
$X \in (2\mathbb{Z} + 0)$
$Y \in (6\mathbb{Z} + 2)$

X is even

gcd(18,30)
Y can be incremented by 18
or decremented by 30 at any iteration

Note that, except when an abstract value represents a constant, any test of the form $X < 40$ *or* $X > 8$ *is abstracted as the identity.*

Summary

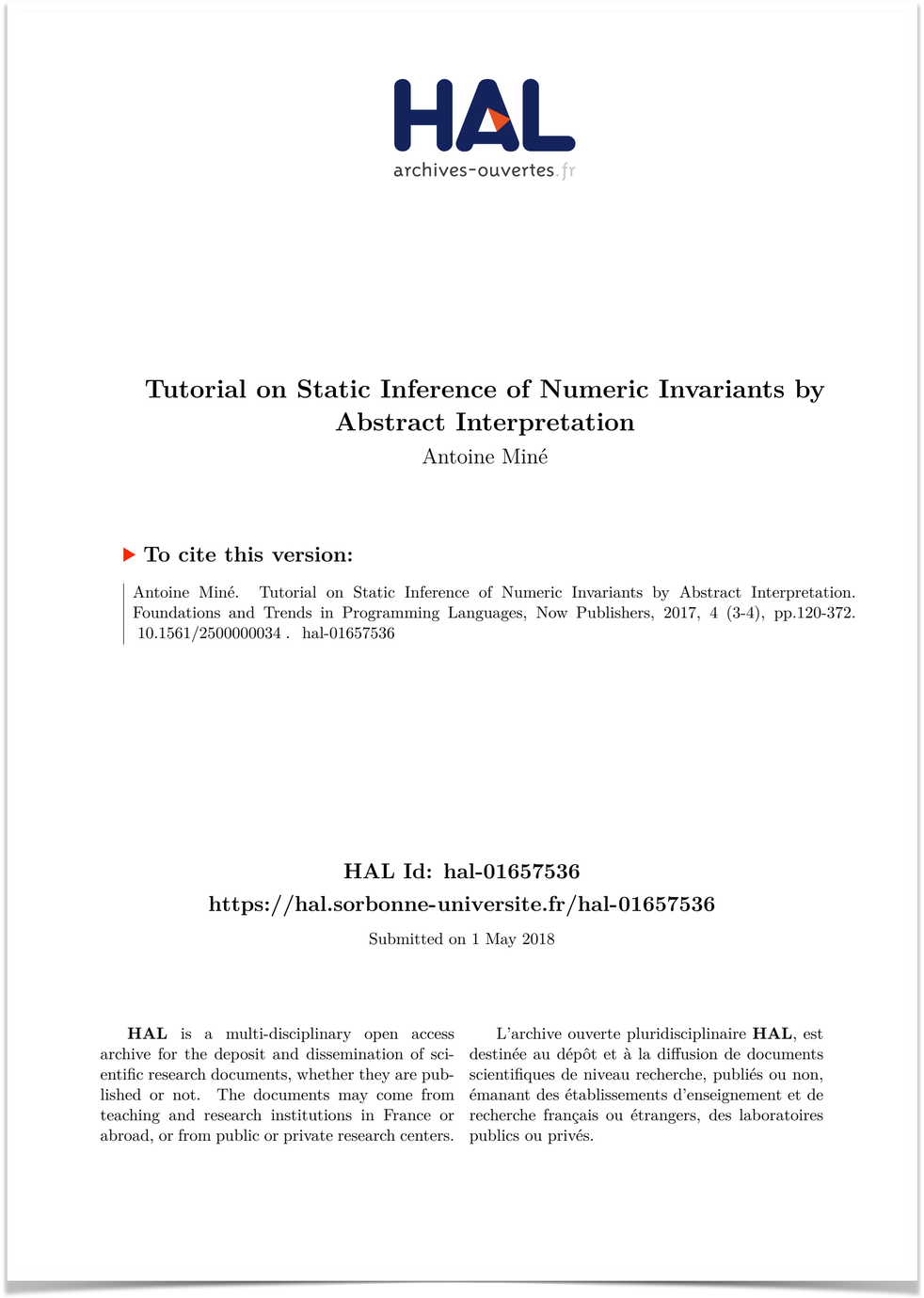


HAL
archives-ouvertes.fr

Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation

Antoine Miné

signs	$0, (\geq 0), (> 0), (\leq 0), (< 0), (\neq 0)$	Sect. 4.2
constants	$c \in \mathbb{Z}$	Sect. 4.3
constant sets	$C \in \mathcal{P}_{finite}(\mathbb{Z})$	Sect. 4.4
intervals	$[a, b]$	Sect. 4.5
congruences	$a\mathbb{Z} + b$	Sect. 4.8



HAL
archives-ouvertes.fr

Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation

Antoine Miné

Overview of Chapter 5: Relational abstract domains

Convex Polyhedra domain

sets of numerical constraints of the form

$$c_1 x + c_2 y \leq c$$

(**at most** two variables per constraint, with **coefficients**)

is a **relational** domain, can be used

when intervals are not expressive enough

Example

$x, y \in [0,5]$

$$
\begin{aligned}
1x + 0y &\leq 5 \\
-1x + 0y &\leq 0 \\
0x + 1y &\leq 5 \\
0x + -1y &\leq 0
\end{aligned}
$$

$x = y$

$$
\begin{aligned}
1x + -1y &\leq 0 \\
-1x + 1y &\leq 0
\end{aligned}
$$

$0 \leq x < y \leq 5$

$$
\begin{aligned}
-1x + 0y &\leq 0 \\
-1x + 1y &\leq 1 \\
0x + 1y &\leq 5
\end{aligned}
$$

Example

$x, y \in [0,5]$

$$
\begin{aligned}
1x &\leq 5 \\
-1x &\leq 0 \\
1y &\leq 5 \\
-1y &\leq 0
\end{aligned}
$$

$x = y$

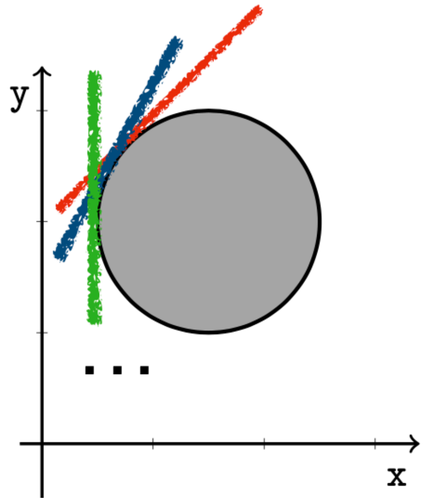
$$
\begin{aligned}
1x + -1y &\leq 0 \\
-1x + 1y &\leq 0
\end{aligned}
$$

$0 \leq x < y \leq 5$

$$
\begin{aligned}
-1x &\leq 0 \\
-1x + 1y &\leq 1 \\
1y &\leq 5
\end{aligned}
$$

No best approximation

Computing the best approximation is not always possible



In practice we can use abstractions

as precise as possible, but maybe not the best

(still best abstractions can exist for some sets of states)

Why is the Polyhedra Domain Expensive?

Combinatorial explosion

- number of constraints can be exponential
- H-representation ↔ V-representation is costly

Expensive operations

- Join (convex hull) → may generate many constraints
- Projection → exponential blow-up (Fourier–Motzkin)

Costly normalization

- removing redundant constraints
- requires linear programming

Inclusion checks

- need to solve LP problems

Octagons domain

sets of numerical constraints of the form

$$\pm x \pm y \leq c$$

(**at most** two variables per constraint, with **unit** coefficients)

is a **relational** domain, can be used

when intervals are not expressive enough

and polyhedra are more costly

(allow trading precision for efficiency)

Example

$x, y \in [0,5]$

$$
\begin{aligned}
1x &\leq 5 \\
-1x &\leq 0 \\
1y &\leq 5 \\
-1y &\leq 0
\end{aligned}
$$

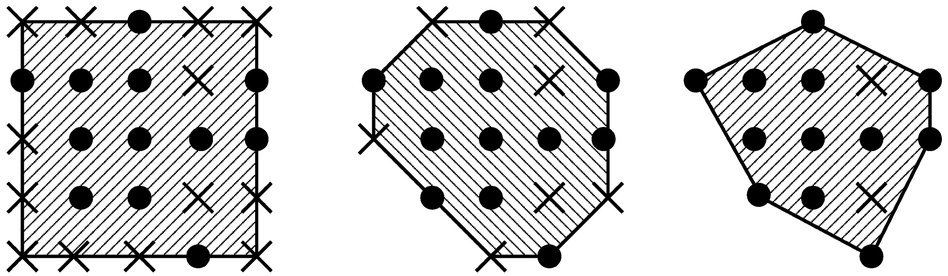
$x = y$

$$
\begin{aligned}
1x + -1y &\leq 0 \\
-1x + 1y &\leq 0
\end{aligned}
$$

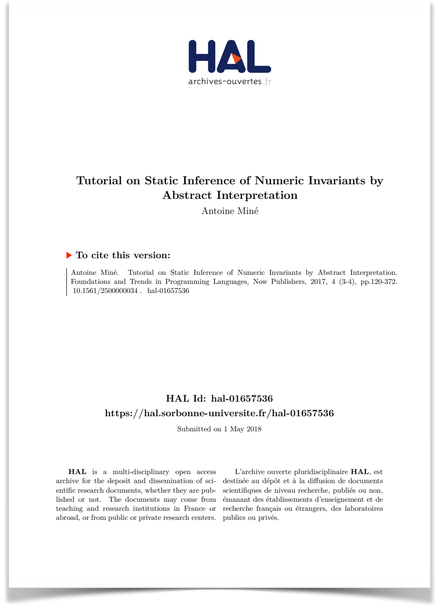
$0 \leq x < y \leq 5$

$$
\begin{aligned}
-1x &\leq 0 \\
-1x + 1y &\leq 1 \\
1y &\leq 5
\end{aligned}
$$

Summary



Intervals Octagons Polyhedra



HAL
archives-ouvertes.fr

Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation

Antoine Miné

To cite this version:

Antoine Miné. Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation. Foundations and Trends in Programming Languages, Now Publishers, 2017, 4 (3-4), pp.120-372. 10.1561/2500000034 . hal-01657536

HAL Id: hal-01657536
https://hal.sorbonne-universite.fr/hal-01657536

Submitted on 1 May 2018

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire HAL, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

domain	invariants	memory cost	time cost	Sect.
affine equalities	$\sum_i \alpha_i V_i = \beta$	$\mathcal{O}(\lvert\mathbb{V}\rvert^2)$	$\mathcal{O}(\lvert\mathbb{V}\rvert^3)$	5.2
zones	$V_j - V_i \leq c$	$\mathcal{O}(\lvert\mathbb{V}\rvert^2)$	$\mathcal{O}(\lvert\mathbb{V}\rvert^3)$	5.4
octagons	$\pm V_j \pm V_i \leq c$	$\mathcal{O}(\lvert\mathbb{V}\rvert^2)$	$\mathcal{O}(\lvert\mathbb{V}\rvert^3)$	5.4.5
template	$\sum_i \alpha_i V_i \geq \beta$	$\mathcal{O}(m)$	polynomial	5.5
polyhedra	$\sum_i \alpha_i V_i \geq \beta$	exponential	exponential	5.3

fixed coefficients

affine inequalities

Exercises

Classification

signs

parity

intervals around 0 ($[-l, r]$)

equalities ($c_1 x = c_2 y$)

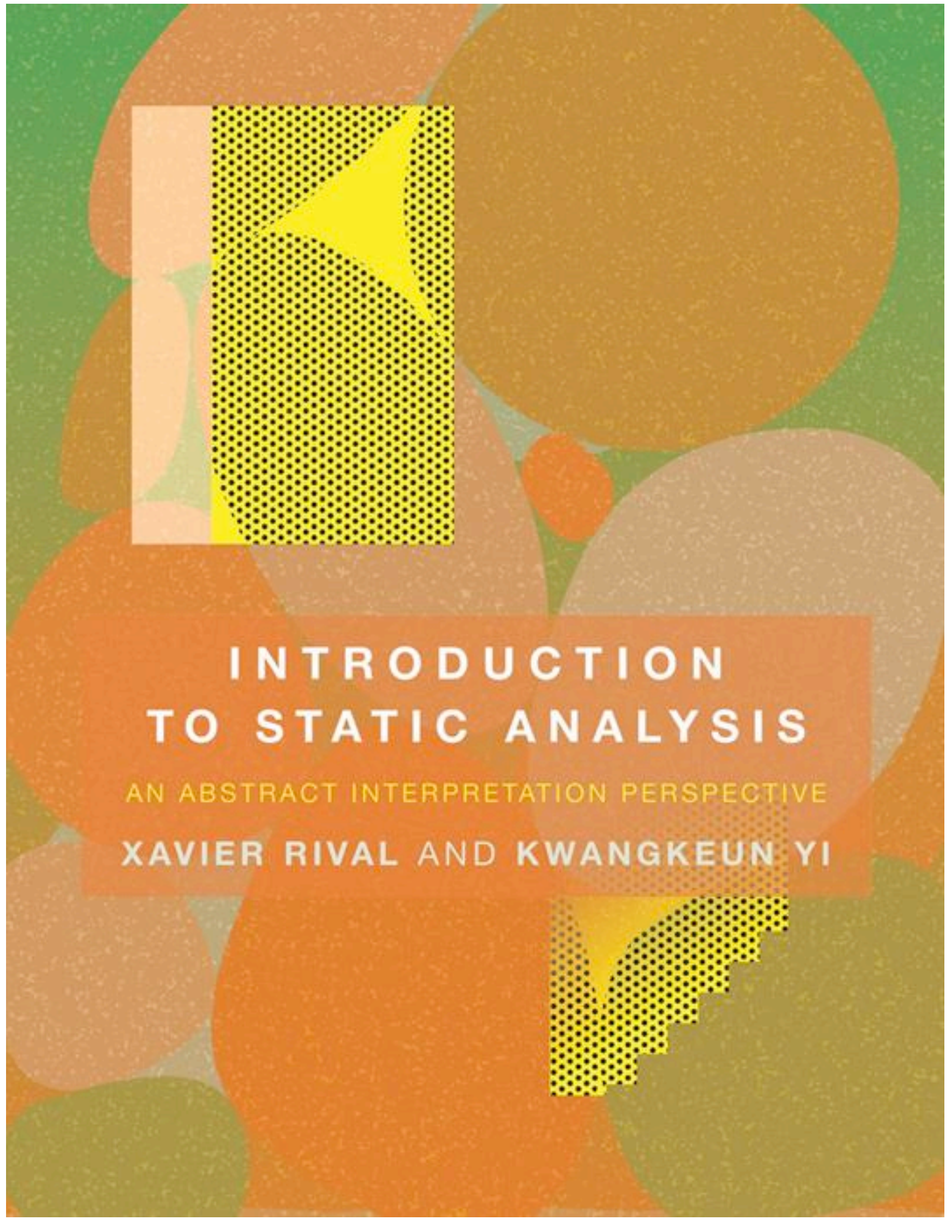
inequalities ($c_1 x \neq c_2 y$)

relational

or

non relational?

Deriving new lattices



Section 5.1:
Construction of abstract domains

Composition

$$(A_0, \sqsubseteq_0) \underset{\alpha_1}{\overset{\gamma_1}{\leftrightarrows}} (A_1, \sqsubseteq_1) \qquad (A_1, \sqsubseteq_1) \underset{\alpha_2}{\overset{\gamma_2}{\leftrightarrows}} (A_2, \sqsubseteq_2)$$

the composition of two GCs is a GC

$$(A_0, \sqsubseteq_0) \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} (A_2, \sqsubseteq_2)$$

$$\gamma = \gamma_1 \circ \gamma_2$$

$$\alpha = \alpha_2 \circ \alpha_1$$

Conjoint analysis: product domains

Conjunctive properties

program verification often requires the use of logical predicates that correspond to the conjunction of several basic predicates

concrete domain = powerset of integers {all possible subset}

abstract domain = product domain:

intervals abstraction × congruence abstraction

Cartesian product

Given the (complete) lattices $(L_i , \sqsubseteq_i , \sqcup_i , \sqcap_i , \bot_i , \top_i)$ for $i = 1,2$

We derive their **cartesian product** lattice $(L, \sqsubseteq, \sqcup, \sqcap, \bot, \top)$ by taking:

- $L \triangleq L_1 \times L_2$;
- $(a_1, a_2) \sqsubseteq (b_1, b_2)$ iff $a_1 \sqsubseteq_1 b_1$ and $a_2 \sqsubseteq_2 b_2$;
- $(a_1, a_2) \sqcup (b_1, b_2) \triangleq (a_1 \sqcup_1 b_1 , a_2 \sqcup_2 b_2)$;
- $(a_1, a_2) \sqcap (b_1, b_2) \triangleq (a_1 \sqcap_1 b_1 , a_2 \sqcap_2 b_2)$;
- $\bot \triangleq (\bot_1 , \bot_2)$;
- $\top \triangleq (\top_1 , \top_2)$.

Cartesian product domain

$$(C, \leq) \underset{\alpha_1}{\overset{\gamma_1}{\leftrightarrows}} (A_1, \sqsubseteq_1) \qquad (C, \leq) \underset{\alpha_2}{\overset{\gamma_2}{\leftrightarrows}} (A_2, \sqsubseteq_2)$$

the product of two GCs is a GC

$$(C, \leq) \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} (A_1 \times A_2, \sqsubseteq)$$

$$\gamma(a_1, a_2) = \gamma_1(a_1) \wedge \gamma_2(a_2)$$

Conjunctive properties

e.g. the abstract state ($[0,100]$, $2\mathbb{Z}$)

describes the set

$$\gamma(\ [0,100]\ ,\ 2\mathbb{Z}\ )$$

$$= \{0,...,100\} \cap \{\ldots, -2,0,2,...\}$$

$$= \{0,2,4,...,100\}$$

all even values between 0 and 100

Problem: non injective γ

consider $([1,5] , 2\mathbb{Z}) \neq ([2,4] , 2\mathbb{Z})$

both represent the same concrete set $\{2,4\}$!

consider $(\bot , 2\mathbb{Z})$, $([2,4] , \bot)$ and $(\bot , \bot)$

they all represent the same concrete set $\emptyset$!

solution: reduced product!

Reduced product domain

$$(C, \leq) \underset{\alpha_1}{\overset{\gamma_1}{\leftrightarrows}} (A_1, \sqsubseteq_1) \qquad (C, \leq) \underset{\alpha_2}{\overset{\gamma_2}{\leftrightarrows}} (A_2, \sqsubseteq_2)$$

the reduced product of two GCs is a GC

$$(C, \leq) \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} ((A_1 \times A_2)_{\equiv}, \sqsubseteq)$$

take the equivalence classes

$$(a_1, a_2) \equiv (b_1, b_2) \Leftrightarrow \gamma(a_1, a_2) = \gamma(b_1, b_2)$$

$$\gamma([a_1, a_2]_{\equiv}) = \gamma_1(a_1) \cap \gamma_2(a_2)$$

Reduced product domain

$$(C, \leq) \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} ((A_1 \times A_2)_{\equiv}, \sqsubseteq)$$

take the equivalence classes

$$(a_1, a_2) \equiv (b_1, b_2) \Leftrightarrow \gamma(a_1, a_2) = \gamma(b_1, b_2)$$

on this domain $([1,5] , 2\mathbb{Z}) \equiv ([2,4] , 2\mathbb{Z})$

both represent the same concrete set $\{2,4\}$!

on this domain $(\bot , 2\mathbb{Z}) \equiv ([2,4] , \bot) \equiv (\bot , \bot)$

all represent the same concrete set $\varnothing$!

Point-wise lifting

Point-wise lifting

Given the (complete) lattice $(L_1 , \sqsubseteq_1 , \sqcup_1 , \sqcap_1 , \bot_1 , \top_1)$ and a set X

We derive its **pointwise lifting** lattice $(L, \sqsubseteq, \sqcup, \sqcap, \bot, \top)$ by taking:

- $L \triangleq X \to L_1$;
- $f \sqsubseteq g$ iff $\forall x \in X : f(x) \sqsubseteq_1 g(x)$;
- $(f \sqcup g) \triangleq \lambda x \,.\, f(x) \sqcup_1 g(x)$;
- $(f \sqcap g) \triangleq \lambda x \,.\, f(x) \sqcap_1 g(x)$;
- $\bot \triangleq \lambda x \,.\, \bot_1$;
- $\top \triangleq \lambda x \,.\, \top_1$.

$x, y \in X, l \in L_1 \backslash \{ \bot_{L_1} \}$ we can have $\{ x \mapsto \bot_{L_1} , y \mapsto l \}$

Smashed point-wise lifting

Given the (complete) lattice $(L_1 , \sqsubseteq_1 , \sqcup_1 , \sqcap_1 , \bot_1 , \top_1)$ and a set X

We derive its **smashed pointwise lifting** lattice $(L, \sqsubseteq, \sqcup, \sqcap, \bot, \top)$ by taking:

- $L = X \dot{\rightarrow} L_1 \triangleq (X \rightarrow (L_1 \backslash \{ \bot_1 \})) \uplus \{ \lambda x \,.\, \bot_1 \}$;
- $f \sqsubseteq g$ iff $\forall x \in X : f(x) \sqsubseteq_1 g(x)$;
- $(f \sqcup g) \triangleq \lambda x \,.\, f(x) \sqcup_1 g(x)$;
- $(f \sqcap g) \triangleq \lambda x \,.\, f(x) \sqcap_1 g(x)$;
- $\bot \triangleq \lambda x \,.\, \bot_1$;
- $\top \triangleq \lambda x \,.\, \top_1$.

$x, y \in X$, we can only have $\{ x \mapsto \bot_{L_1} , y \mapsto \bot_{L_1} \}$

Smashed point-wise lifting domain

$$(C, \leq) \underset{\alpha_1}{\overset{\gamma_1}{\leftrightarrows}} (A_1, \sqsubseteq_1)$$

$$(X \to C, \leq) \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} (X \dot{\to} A_1, \sqsubseteq)$$

smashed

$$\gamma(f) = \lambda x \,.\, \gamma_1(f(x))$$

Example

$$(\wp(\mathbb{Z}), \subseteq) \underset{\alpha_1}{\overset{\gamma_1}{\leftrightarrows}} (\mathsf{Int}, \sqsubseteq_1)$$

$$(X \to \wp(\mathbb{Z}), \subseteq) \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} (X \dot{\to} \mathsf{Int}, \sqsubseteq)$$

non-empty intervals for all variables
or
the empty interval for all variabkles

$$\gamma(f) = \lambda x \,.\, \gamma_1(f(x))$$

Abstract stores

Abstract stores

let $\Sigma \triangleq \{\sigma \colon X \to \mathbb{Z}\}$ be the set of stores

concrete domain = powerset of store $\wp(\Sigma)$

$$(\wp(\mathbb{Z}), \subseteq) \underset{\alpha_1}{\overset{\gamma_1}{\leftrightarrows}} (A_1, \sqsubseteq_1)$$

first: (smashed) point-wise abstraction (cartesian abstraction)

from $\wp(\Sigma)$ to $\{\rho : X \to \wp(\mathbb{Z})\}$ (looses relational info!)

second: lifting the abstraction A_1

from $\{\rho : X \to \wp(\mathbb{Z})\}$ to $\{\rho^{\#} : X \to A_1\}$

First: Cartesian abstraction

from $\wp(X \to \mathbb{Z})$ to $X \to \wp(\mathbb{Z})$ (looses all relational info!)

$$\gamma(\rho) = \{\ \sigma : X \to \mathbb{Z} \mid \forall x \in X\,.\ \sigma(x) \in \rho(x)\ \}$$

e.g., suppose $\rho = [x \mapsto \{0,5\}, y \mapsto \{0,3\}]$

$$\gamma(\rho) = \{\ [x \mapsto 0, y \mapsto 0], [x \mapsto 0, y \mapsto 3], [x \mapsto 5, y \mapsto 0], [x \mapsto 5, y \mapsto 3]\ \}$$

$$\alpha(\{[x \mapsto 0, y \mapsto 0], [x \mapsto 5, y \mapsto 3]\}) = \rho$$

Second: lifting abstraction

$$(\wp(\mathbb{Z}), \subseteq) \underset{\alpha_1}{\overset{\gamma_1}{\leftrightarrows}} (A_1, \sqsubseteq_1)$$

from $X \to \wp(\mathbb{Z})$ to $\{\rho^{\#} : X \to A_1\}$

$$\gamma(\rho^{\#}) = \{\ \rho : X \to \wp(\mathbb{Z}) \mid \forall x \in X .\ \rho(x) = \gamma_1(\rho^{\#}(x))\ \}$$

e.g., suppose $\rho^{\#} = [x \mapsto [0,5], y \mapsto [0,3]]$

$$\gamma(\rho^{\#}) = [x \mapsto \{0,1,2,...,5\}, y \mapsto \{0,1,2,3\}]$$

$$\alpha([x \mapsto \{0,5\}, y \mapsto \{0,3\}]) = \rho^{\#}$$

Abstract denotational semantics

Abstract interpreter

$[\![e]\!] \triangleq \ldots$

$[\![c_1; c_2]\!] \triangleq [\![c_2]\!] \circ [\![c_1]\!]$

$[\![c_1 + c_2]\!] \triangleq [\![c_1]\!] \cup [\![c_2]\!]$

$[\![c^\star]\!] \triangleq \bigcup_{k=0}^{\infty} [\![c]\!]^k$

or equivalently

$[\![c^\star]\!] \triangleq \lambda S\,.\,\mathsf{lfp}\ \lambda X\,.\,S \cup [\![c]\!]X$

$[\![e]\!]^\# \triangleq [\![e]\!]^A$ (when possible)

$[\![c_1; c_2]\!]^\# \triangleq [\![c_2]\!]^\# \circ [\![c_1]\!]^\#$

$[\![c_1 + c_2]\!]^\# \triangleq [\![c_1]\!]^\# \sqcup [\![c_2]\!]^\#$

$[\![c^\star]\!]^\# \triangleq \bigsqcup_{k=0}^{\infty} ([\![c]\!]^\#)^k$

or, not equivalently,

$[\![c^\star]\!]^\# \triangleq \lambda S^\#\,.\,\mathsf{lfp}\ \lambda X^\#\,.\,S^\# \sqcup [\![c]\!]^\# X^\#$

Accelerated abstract interpreter

$[\![e]\!] \triangleq \ldots$

$[\![c_1; c_2]\!] \triangleq [\![c_2]\!] \circ [\![c_1]\!]$

$[\![c_1 + c_2]\!] \triangleq [\![c_1]\!] \cup [\![c_2]\!]$

$[\![c^\star]\!] \triangleq \bigcup_{k=0}^{\infty} [\![c]\!]^k$

or equivalently

$[\![c^\star]\!] \triangleq \lambda S \,.\, \mathrm{lfp}\ \lambda X \,.\, S \cup [\![c]\!]X$

$[\![e]\!]^\# \triangleq [\![e]\!]^A$ (when possible)

$[\![c_1; c_2]\!]^\# \triangleq [\![c_2]\!]^\# \circ [\![c_1]\!]^\#$

$[\![c_1 + c_2]\!]^\# \triangleq [\![c_1]\!]^\# \sqcup [\![c_2]\!]^\#$

$[\![c^\star]\!]^\# \triangleq \bigsqcup_{k=0}^{\infty} ([\![c]\!]^\#)^k$

or using widening (precision loss)

$[\![c^\star]\!]^\# \triangleq \lambda S^\# \,.\, \mathrm{lfp}\ \lambda X^\# \,.\, S^\# \nabla [\![c]\!]^\# X^\#$

Take home message

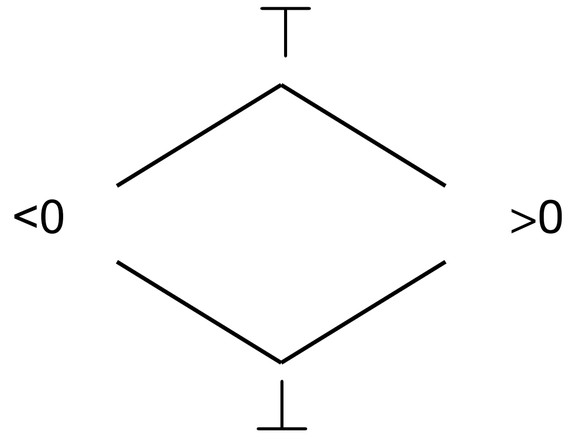
The abstract interpreter is induced by atomic operations

Completeness

Correctness

The abstract operations $+^{\#}$ and $\times^{\#}$ are correct on the domain Sign:

$$\forall n, m \in C\,.\; \alpha(n) +^{\#} \alpha(m) \sqsupseteq \alpha(n + m)$$



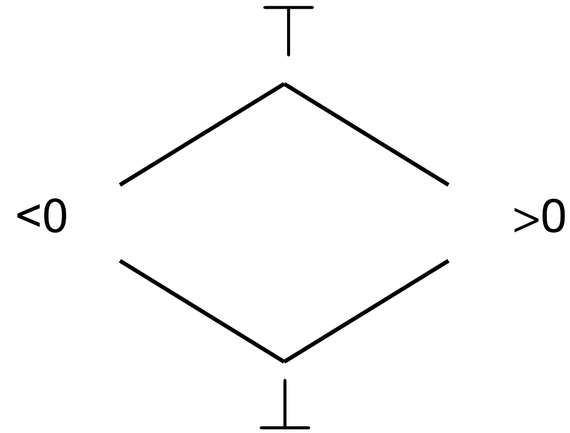
$F^{\#}$ is correct on an abstract domain A whenever it returns a correct approximation of the result of the concrete computation:

$$F^{\#} \sqsupseteq \alpha \circ F \circ \gamma = F^{A}$$

Completeness

The abstract operation $\times^{\#}$ has a very nice property on the domain Sign:

$$\forall n, m \in C \,.\, \alpha(n) \times^{\#} \alpha(m) \trianglerighteq \alpha(n \times m)$$



$F^{\#}$ is complete on an abstract domain A whenever it returns the best abstraction of the result of the concrete computation:

$$F^{\#} \circ \alpha = \alpha \circ F$$

Completeness and bcas

$F^{\#}$ is complete $\Longrightarrow$ $F^{\#} = F^{A}$

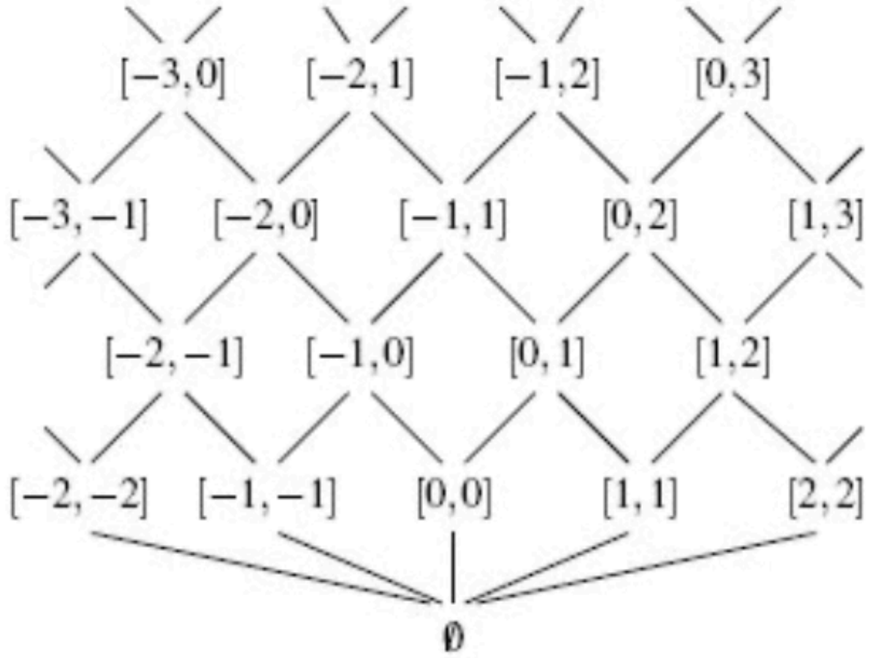
$$\alpha \circ F = F^{\#} \circ \alpha \Longrightarrow F^{A} = \alpha F \gamma = F^{\#} \circ \alpha \circ \gamma = F^{\#}$$

$+^A$ and $\times^A$ are complete on Int

$$[n, m] +^A [p, r] = [n + p, m + r]$$

$$[n, m] \times^A [p, r] = [n \times p, m \times r]$$

if all positives, otherwise pay attention



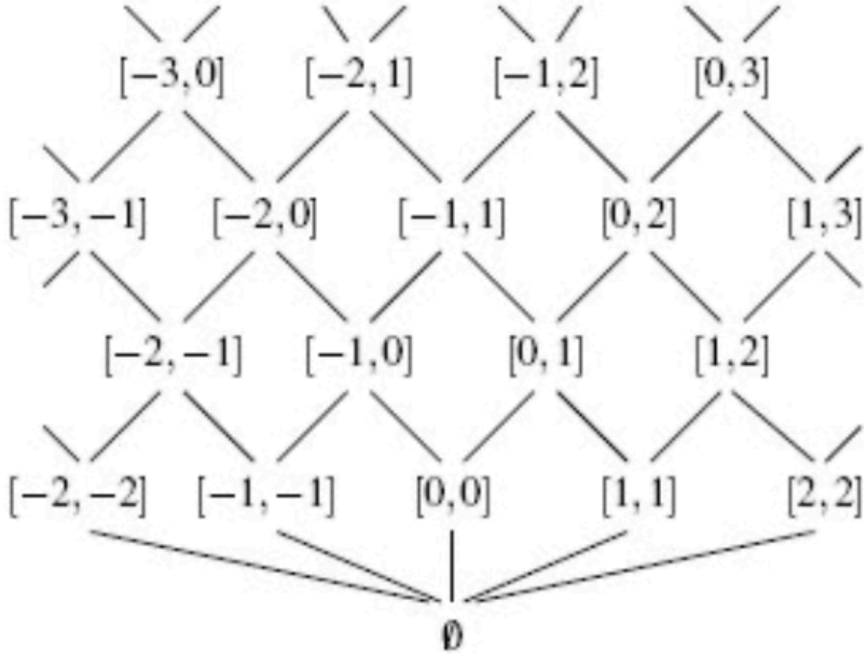
$[1,6]$ $[-3,1]$ $[-2,7]$

$$\{1,4,6\} + \{-3,1\} = \{-2,1,2,3,5,7\}$$

Precise result!

$|\cdot|$ is not complete on Int

$$|[a,b]|^A = \begin{cases} [a,b] & \textbf{if } a \geq 0 \\ [-b,-a] & \textbf{if } b \leq 0 \\ [0,\max(-a,b)] & \textbf{if } a < 0 < b \end{cases}$$



$[-6,1]$ $[0,6]$

$$|\{-6,-4,1\}| = \{1,4,6\}$$

Imprecise result!

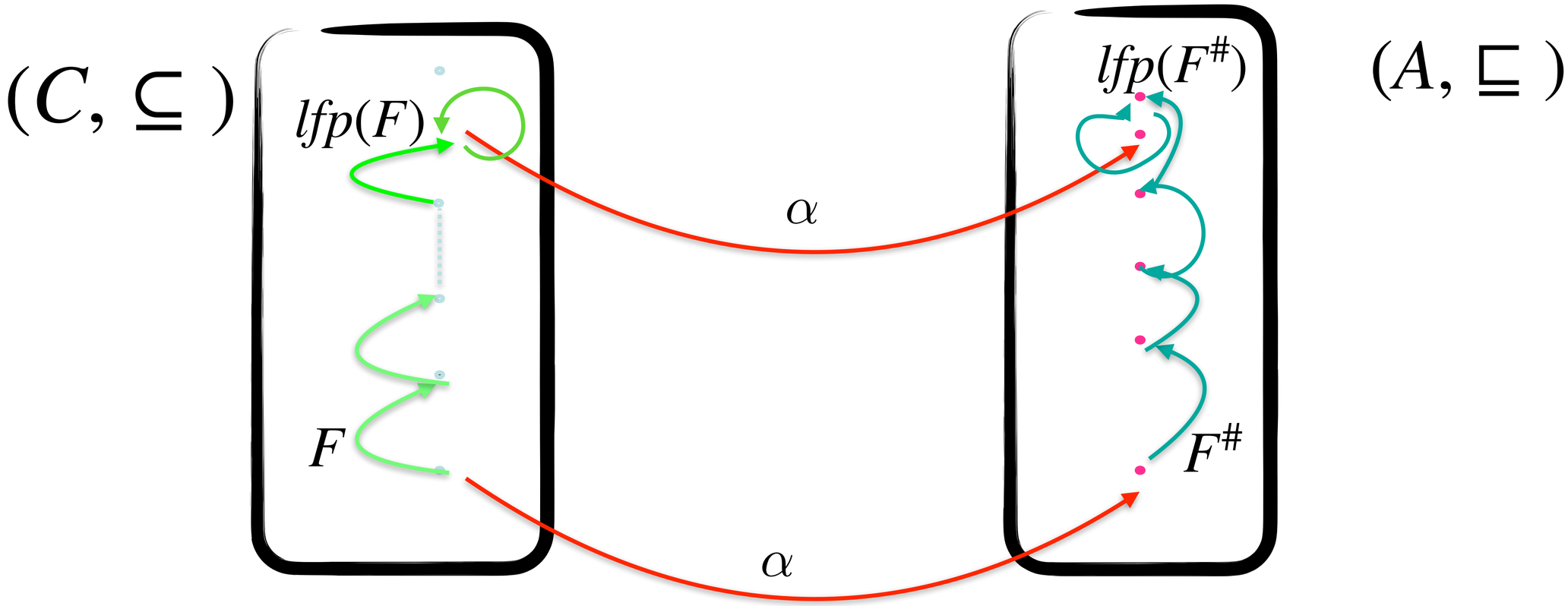
$[-6,-1]$ $[1,6]$

$$|\{-6,-4,-1\}| = \{1,4,6\}$$

Precise result!

Fixpoint computation approximation

If F monotone and $F^{\#}$ correct

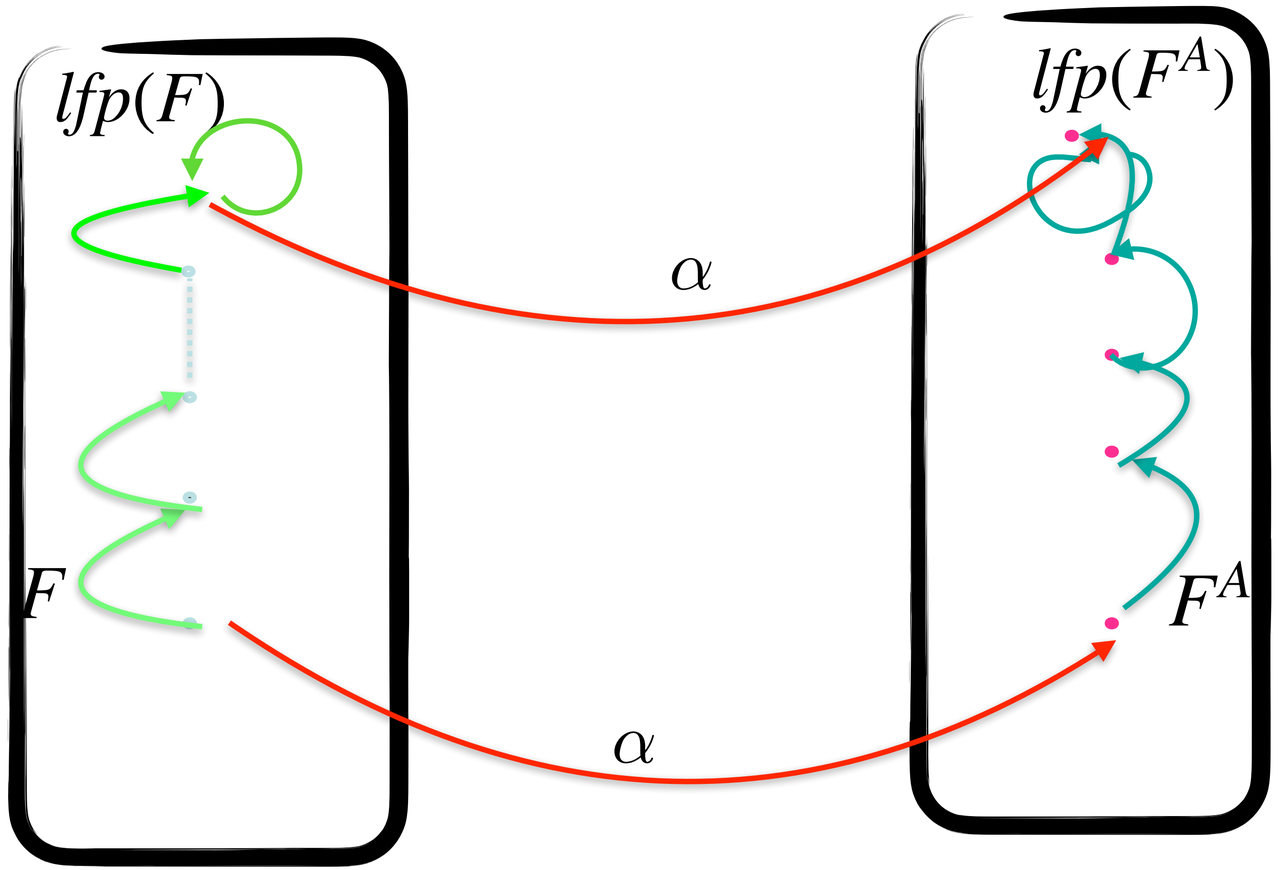


$lfp(F^{\#})$ is a correct over approximation of $lfp(F)$

Fixpoint computation approximation

If F monotone and F^A is complete

$(C, \sqsubseteq)$



$(A, \sqsubseteq)$

Let us analyse a code fragment on Int

```
if (x>7) {
  y := x-7
} else {
  y := 7-x
}
```

{ y ≥ 0 }?

$[\mathtt{x} \mapsto \top\ ,\ \mathtt{y} \mapsto \top\ ]$

$(x > 7)?;$

$[\mathtt{x} \mapsto [8,\infty],\ \mathtt{y} \mapsto \top\ ]$

$y := x - 7$

$[\mathtt{x} \mapsto [8,\infty],\ \mathtt{y} \mapsto [1,\infty]\ ]$

$+\ \ (x \leq 7)?;$

$[\mathtt{x} \mapsto [-\infty,7],\ \mathtt{y} \mapsto \top\ ]$

$y := 7 - x$

$[\mathtt{x} \mapsto [-\infty,7],\ \mathtt{y} \mapsto [0,+\infty]\ ]$

$[\mathtt{x} \mapsto \top\ ,\ \mathtt{y} \mapsto [0,+\infty]\ ]$

Example on Interval

c_1

```
x := 10;
while (x>0) {
        x := x-1
    }; { x = 0 }?
```

Complete!

$[\mathtt{x} \mapsto \top\ ]$

$(x := 10);$

$[\mathtt{x} \mapsto [10,10]\ ]$

$((x > 0)?$

$[\mathtt{x} \mapsto [1,10]\ ]$

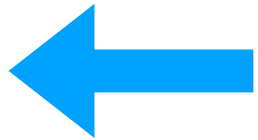
$x := x - 1)^*;$

$[\mathtt{x} \mapsto [0,9]\ ]$

$[\mathtt{x} \mapsto [0,10]\ ]$

$(x \leq 0)?$

$[\mathtt{x} \mapsto [0,0]\ ]$



Abstract loop invariant

Example on Interval

c_2

```
x := 10;
while (x>1) {
      x := x-2
    }; { x = 0 }?
```

Incomplete!

$[\mathtt{x} \mapsto \top]$

$(x := 10);$

$[\mathtt{x} \mapsto [10,10]]$

$((x > 1)?$

$[\mathtt{x} \mapsto [2,10]]$

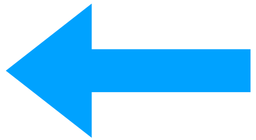
$x := x - 2)^*;$

$[\mathtt{x} \mapsto [0,8]]$

$[\mathtt{x} \mapsto [0,10]]$

$(x \leq 1)?$

$[\mathtt{x} \mapsto [0,1]]$



Abstract loop invariant

Concrete and abstract equivalences

Concrete Equivalence $\approx$:

$$c_1 \approx c_2$$

$\Longleftrightarrow$ They compute the same function

$$\forall S \in \wp(\Sigma) \,.\, [\![c_1]\!]S = [\![c_2]\!]S$$

Abstract Equivalence $\sim$:

$$c_1 \sim c_2$$

$\Longleftrightarrow$ They compute the same abstract function

$$\forall a : X \to A \,.\, [\![c_1]\!]^{\#}a = [\![c_2]\!]^{\#}a$$

Concrete equivalence does not imply Abstract equivalence

c_1

```
x := 10;
while (x>0) {
      x := x-1
    };
```

Complete!

$\approx$

$\not\approx$

c_2

```
x := 10;
while (x>1) {
      x := x-2
    };
```

Incomplete!

$$\forall S. [\![c_1]\!] S = [x \mapsto 0] = [\![c_2]\!] S$$

but

$$\forall a. [\![c_1]\!]^{\#} a = [x \mapsto [0,0]] \neq [\![c_2]\!]^{\#} a = [x \mapsto [0,1]]$$

Abstract equivalence does not imply Concrete equivalence

c_3

```
x := -9;
while (x<0) {
      x := x+2
};
```

$x \to [0,1]$

$\sim$

$\not\sim$

c_4

```
x := 10;
while (x>1) {
      x := x-2
};
```

$x \to [0,1]$

$$\forall a. [\![c_3]\!]^\# a = [x \mapsto [0,1]] = [\![c_4]\!]^\# a$$

but

$$\forall S. [\![c_3]\!] S = [x \mapsto 1] \neq [\![c_4]\!] S = [x \mapsto 0]$$

Abstract property A is Intensional!!!

The precision depends on the way the program is written

Complete!

c_1

```
x := 10;
while (x>0) {
        x := x-1
      }; { x = 0 }
```

$x \in [0,0]$

Non Extensional

Incomplete!

c_2

```
x := 10;
while (x>1) {
        x := x-2
      }; { x = 0 }
```

$x \in [0,\mathbf{1}]$

Abstract analysis

Three staged approach: 1

Selection of the semantics and properties of interest

define a concrete domain $(C, \subseteq)$

define program behaviour $[\![\cdot]\!]$

fix the goal of the analysis

describe properties to be verified

Three staged approach: 2

Choice of the abstraction

fix the abstract domain $(A, \sqsubseteq)$

must express all properties of interest

(must express useful invariants)

define abstract interpreter $[\![\cdot]\!]^{\#}$

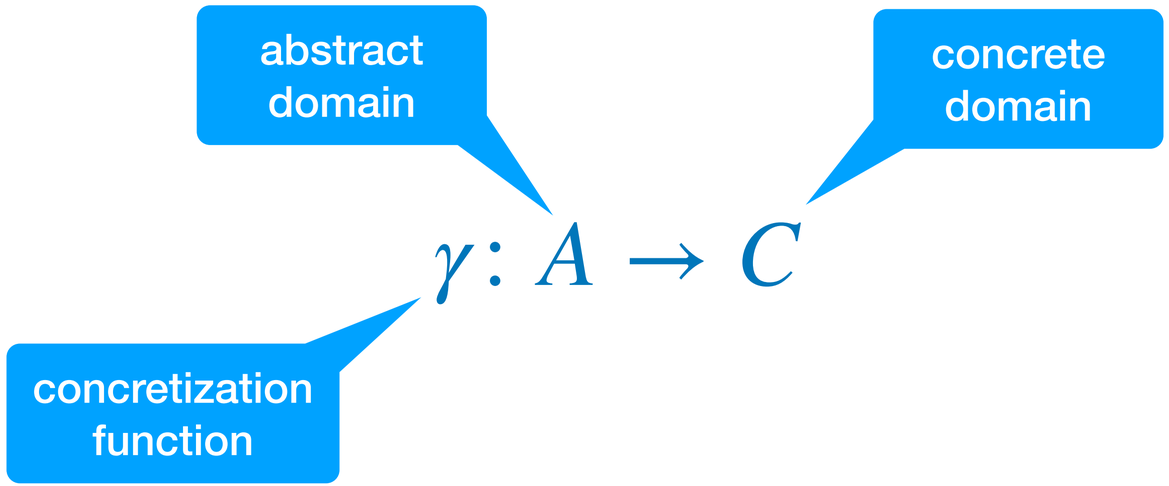
We need the ability to

- compare abstract elements for inclusion
- compute (possibly best) abstract representations
- compute the union/widening of abstract elements
- apply sound transformations to abstract elements

Abstract domain

We need the ability to
compare abstract elements for inclusion

$(A, \sqsubseteq)$

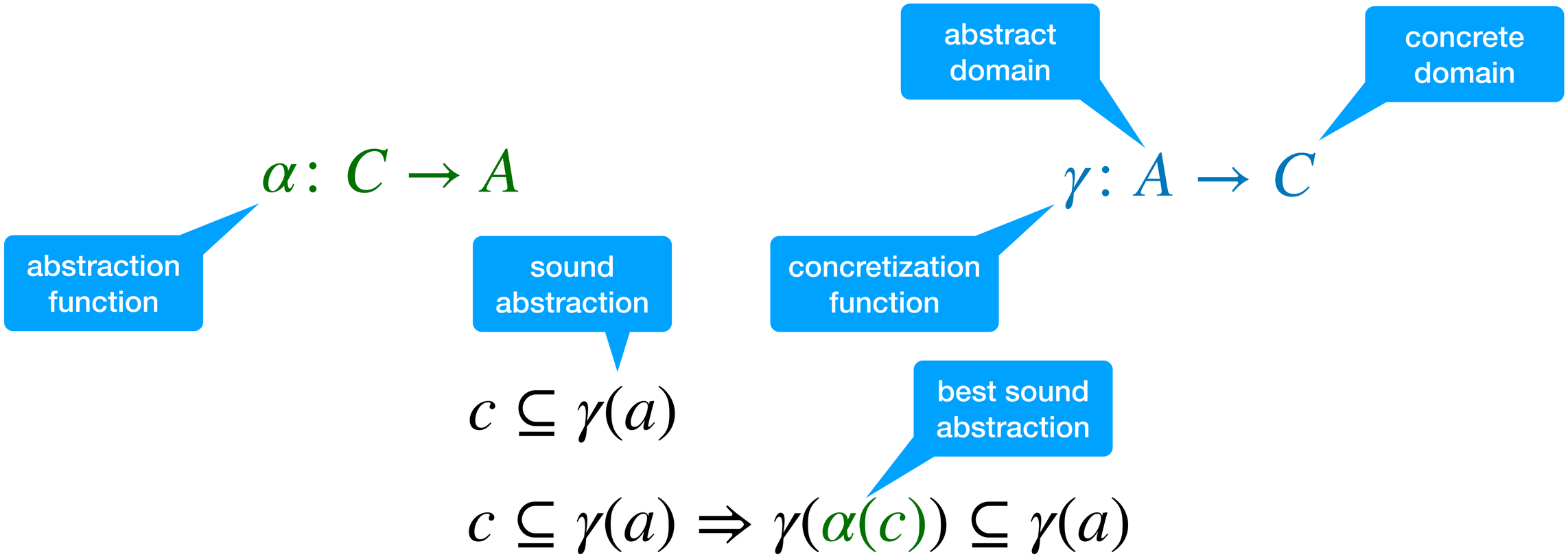


$$a \sqsubseteq b \Rightarrow \gamma(a) \subseteq \gamma(b)$$

Abstract domain

We need the ability to

compute (possibly best) abstract representations



Abstract domain

We need the ability to
compute the union/widening of abstract elements

union

$$a_1, a_2 \sqsubseteq a_1 \sqcup a_2$$

widening

$$a_1 \sqcup a_2 \sqsubseteq a_1 \nabla a_2$$

Abstract domain

We need the ability to

apply sound transformations to abstract elements

concrete function

$$f\colon C \to C$$

abstract function

$$f^{\#}\colon A \to A$$

abstract analysis

sound abstraction

$$f \circ \gamma \subseteq \gamma \circ f^{\#}$$

concretization

$$\begin{array}{ccc} a & \xrightarrow{f^{\#}} & f^{\#}(a) \\ \gamma \downarrow & & \downarrow \gamma \\ \gamma(a) & \xrightarrow[f]{} & f(\gamma(a)) \subseteq \gamma(f^{\#}(a)) \end{array}$$

Three staged approach: 3

Derivation of the analysis algorithm

follow the choices made in first two phases
the analysis closely follows the semantics

Concrete analysis

$R, S \leftarrow \bot$

`repeat`

$R \leftarrow R \cup S$

$S \leftarrow F(S)$

`until` $S \subseteq R$

`return` R

check label-wise inclusion

$S \leftarrow \bot$

`repeat`

$R \leftarrow S$

typically equivalent

$S \leftarrow S \cup F(S)$

`until` $S \subseteq R$

`return` R

check label-wise inclusion

Abstract analysis

$R^\#, S^\# \leftarrow \bot$

repeat

$R^\# \leftarrow R^\# \sqcup S^\#$

$S^\# \leftarrow F^\#(S^\#)$

until $S^\# \sqsubseteq R^\#$

return $R^\#$

check label-wise inclusion

$S^\# \leftarrow \bot$

repeat

$R^\# \leftarrow S^\#$

$S^\# \leftarrow S^\# \sqcup F^\#(S^\#)$

typically over-approximate

until $S^\# \sqsubseteq R^\#$

return $R^\#$

check label-wise inclusion

Accelerated abstract analysis

$R^\#, S^\# \leftarrow \bot$

`repeat`

$R^\# \leftarrow R^\# \sqcup S^\#$

$S^\# \leftarrow F^\#(S^\#)$

`until` $S^\# \sqsubseteq R^\#$

`return` $R^\#$

check label-wise inclusion

$S^\# \leftarrow \bot$

`repeat`

$R^\# \leftarrow S^\#$

$S^\# \leftarrow S^\# \nabla F^\#(S^\#)$

over-approximate to accelerate convergence

`until` $S^\# \sqsubseteq R^\#$

`return` $R^\#$

check label-wise inclusion

Plan B

What if analysis fails (imprecision, scalability matters)?

check if property of interest can be expressed
check if abstraction preserves properties of interest

identify abstract operations responsible of failure
(loss of precision discarding necessary information)

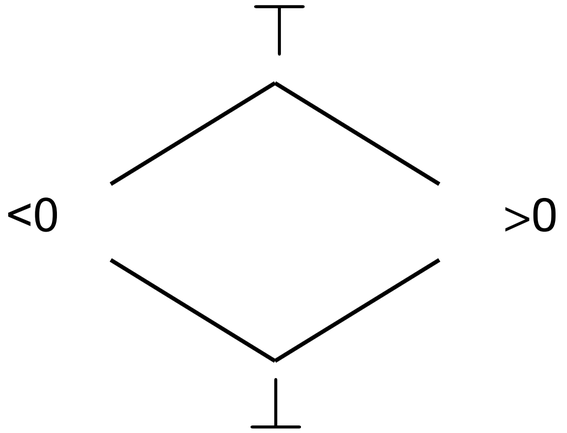
improve analysis parameters/settings if needed

Exercises

Completeness 1

Let $P \triangleq (x \in \{-7,5\})$ and $c \triangleq (x < 0)?; x := -x$.

1. Compute the abstract semantics $[\![c]\!]^{\#}_{\mathsf{Sign}}$ on $\alpha_{\mathsf{Sign}}(P)$
2. Check if the result is the same as $\alpha_{\mathsf{Sign}}([\![c]\!]P)$



Completeness 2

Is the bca of $f : \mathbb{Z} \to \mathbb{Z}$ below complete on the Interval domain?

$$f(x) = \begin{cases} x & \text{if } x \leq 10 \\ 10 & \text{Otherwise} \end{cases}$$

$\alpha(f(X)) \simeq [f(\min X), f(\max X)]$

$f_A(\alpha(f(x)) \supseteq f_A([\min X, \max X]) \simeq$

Completeness 3

Consider the abstract domain Sign′ in the figure

1. Define the corresponding γ and α.

2. Does it admit a complete abstract multiplication?